

PROPOSTA ARQUITETURAL • TCC

Sistema de Autenticação Segura com Múltiplos Fatores

Django • PostgreSQL • Oracle Cloud Infrastructure

NOME DO AUTOR

Curso de Tecnologia da Informação

27 de agosto de 2026



Introdução e objetivos

Senhas isoladas já não são uma fronteira suficiente.

AMEAÇA

Ataques de força bruta, phishing e reutilização de credenciais ampliam o risco.

OBJETIVO

Desenvolver um módulo 2FA seguro com Django e PostgreSQL em uma VPS Oracle Cloud.

ESTRATÉGIA

Aplicar Defense in Depth para mitigar vulnerabilidades do OWASP Top 10 (2021).

Hardening e segurança da infraestrutura

01 SSH por chaves

Ed25519 ou RSA-4096; login root e autenticação por senha desabilitados.

02 Firewall em duas camadas

OCI Security Lists + UFW. Exposição mínima: portas 22, 80 e 443.

03 Bloqueio ativo

Fail2Ban bane IPs após múltiplas tentativas falhas.



Segurança da aplicação e transporte

100%

do tráfego protegido
via HTTPS

TLS + HSTS

Let's Encrypt com renovação e redirecionamento obrigatório.

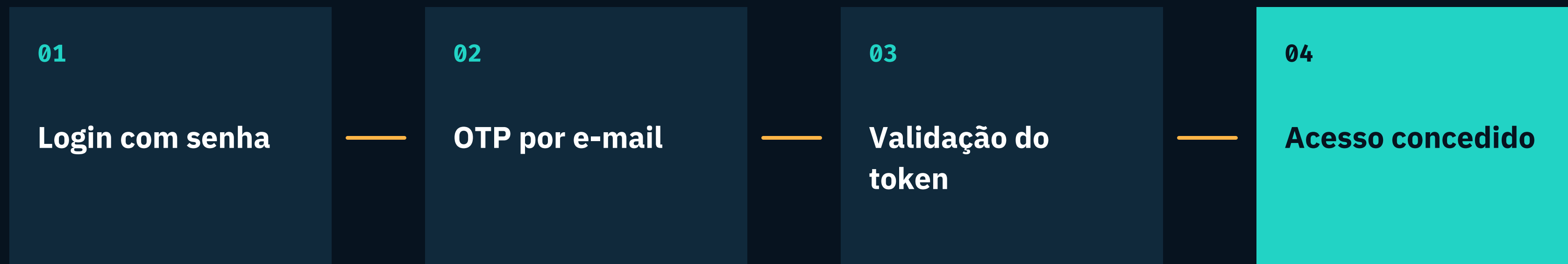
SEGREDOS ISOLADOS

Variáveis de ambiente (.env), fora do código e do Git.

SESSÃO + RATE LIMIT

Cookies HttpOnly e Secure; limitação de taxa contra abuso e força bruta.

Fluxo de autenticação com 2FA



Token temporário expira em 15 minutos • tentativas excessivas acionam rate limiting

Controles vs OWASP Top 10 (2021)

A01	Controle de acesso	Permissões backend + rotas protegidas
A02	Criptografia	TLS, HSTS e cookies Secure
A03	Injeção	ORM Django + validação
A04	Design inseguro	Defense in Depth
A07	Autenticação	2FA, expiração e rate limit
A09	Monitoramento	Logs + Fail2Ban

Controle de acesso e sessões seguras

A01

Cookies Secure + HttpOnly

Reduzem o risco de roubo de sessão e acesso por scripts.

Rotas e permissões no backend

Usuários não autenticados ou não ativados são bloqueados no Django.

```
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
@login_required
```

Criptografia e proteção em trânsito

CLIENTE → TLS 1.2+ → NGINX → DJANGO

Certificados legítimos

Let's Encrypt e renovação automática.

HSTS obrigatório

Força HTTPS e evita downgrade attacks.

Tokens protegidos

Confidencialidade e integridade para senhas e OTPs.

A02

Prevenção contra injeção

```
User.objects.filter(  
    email=email,  
    is_active=True  
)
```

ENTRADA $\neq$ **QUERY SQL**

O ORM parametriza as consultas.

Dados do usuário são tratados como valores — não como comandos. Isso reduz SQL Injection em formulários e APIs. Validação e higienização no backend completam a barreira.

DevSecOps e controle de versão seguro

01

ISOLAR

Segredos em .env; nunca no repositório.

02

IGNORAR

.gitignore rigoroso para dados sensíveis.

03

AUDITAR

Revisão contínua de código e dependências.

04

AUTOMATIZAR

Scanners de vulnerabilidade no pipeline.

Visão geral: Defense in Depth

Uma falha não
compromete o todo.

- 01 INFRAESTRUTURA
- 02 TRANSPORTE
- 03 APLICAÇÃO + 2FA
- 04 MONITORAMENTO

Controles sobrepostos criam redundância e reduzem o impacto de vulnerabilidades isoladas.



Conclusão e próximos passos

2FA + hardening transformam autenticação em um sistema de camadas — não em uma única barreira.

VALIDADO

Controles alinhados ao OWASP Top 10 e à proteção de credenciais.

EVOLUIR

App autenticador, monitoramento avançado e testes de penetração.

PERGUNTAS?

Obrigado.